

Stéphane Quentin
Communiquer, c'est rayonner !

Tutoriel OpenClaw

Maîtrisez le développement d'applications assistées
par IA

Version 2026

Par Stéphane Quentin

Tutoriel Complet : Configurer OpenClaw pour le Développement d'Applications

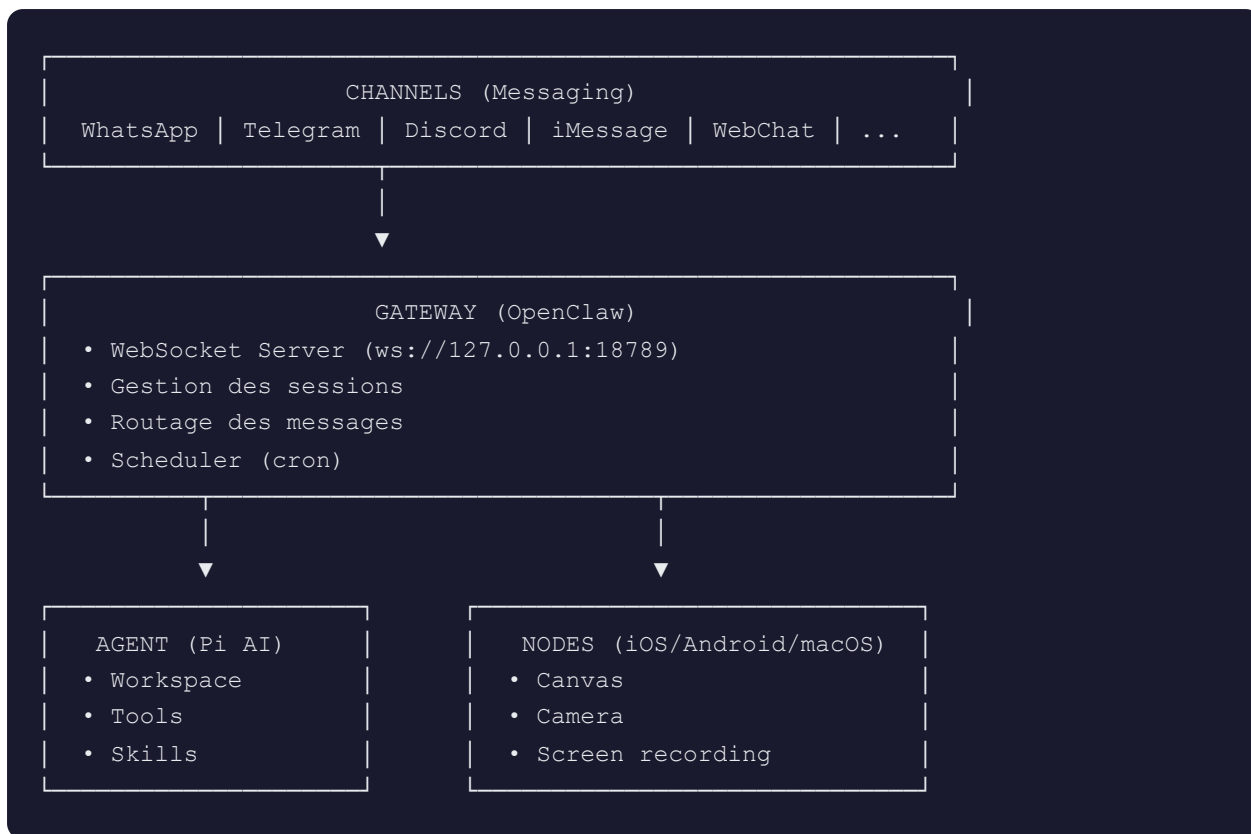
Objectif : Maîtriser OpenClaw comme environnement de développement assisté par IA, de l'installation à la production.

Table des Matières

1. [Architecture & Concepts Fondamentaux](#)
 2. [Installation & Configuration Initiale](#)
 3. [Structure du Workspace](#)
 4. [Configuration Avancée du Gateway](#)
 5. [Développement de Skills](#)
 6. [Gestion des Sessions & Agents](#)
 7. [Intégrations Channels](#)
 8. [Sécurité & Sandbox](#)
 9. [Automatisations & Cron](#)
 10. [Bonnes Pratiques & Debugging](#)
-

1. Architecture & Concepts Fondamentaux

1.1 Vue d'ensemble

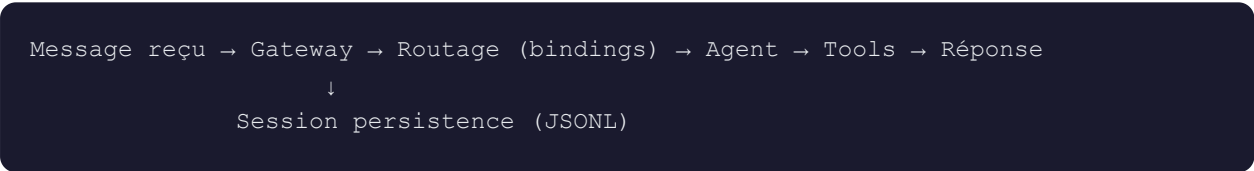


1.2 Composants Clés

Composant	Description	Localisation
Gateway	Processus central qui gère toutes les connexions	<code>~/.openclaw/</code>
Agent	Le "cerveau" IA qui exécute les tâches	Workspace + <code>agentDir</code>
Workspace	Répertoire de travail par défaut	<code>~/.openclaw/workspace</code>
Session	Contexte conversationnel persistant	<code>~/.openclaw/agents/<id>/sessions/</code>

Composant	Description	Localisation
Skills	Extensions qui ajoutent des capacités	<code>~/.openclaw/skills/</code> + workspace
Channels	Interfaces de messagerie (WhatsApp, Telegram, etc.)	Config dans <code>openclaw.json</code>

1.3 Flux de Données



2. Installation & Configuration Initiale

2.1 Prérequis

```
# Node.js ≥ 22 requis
node --version # v22.x.x ou supérieur

# pnpm recommandé (optionnel)
pnpm --version
```

2.2 Installation

```
# Méthode 1 : Script d'installation (recommandé)
curl -fsSL https://openclaw.bot/install.sh | bash

# Méthode 2 : npm global
npm install -g openclaw@latest
```

```
# Méthode 3 : pnpm global
pnpm add -g openclaw@latest
```

2.3 Onboarding Wizard

```
# Lancer l'assistant de configuration complet
openclaw onboard --install-daemon
```

L'assistant configure :

- ☒ Authentification (OAuth ou clés API)
- ☒ Gateway (port, token de sécurité)
- ☒ Channels (WhatsApp, Telegram, Discord...)
- ☒ Service en arrière-plan (systemd/launchd)
- ☒ Workspace de base

2.4 Vérification Post-Install

```
# Vérifier le statut
gateway
openclaw status
openclaw health

# Vérifier la sécurité
openclaw security audit --deep

# Ouvrir le dashboard
openclaw dashboard
# ou http://127.0.0.1:18789/
```

3. Structure du Workspace

3.1 Fichiers Essentiels

Le workspace (`~/.openclaw/workspace` par défaut) contient :

```

~/.openclaw/workspace/
├── AGENTS.md          # Instructions opérationnelles pour l'agent
├── SOUL.md            # Personnalité, ton, boundaries
├── USER.md           # Informations sur l'utilisateur
├── IDENTITY.md        # Identité de l'agent (nom, emoji)
├── TOOLS.md           # Notes sur les outils locaux
├── HEARTBEAT.md       # Checklist pour heartbeats
├── MEMORY.md          # Mémoire long terme (main session uniquement)
├── BOOTSTRAP.md       # Rituel de première connexion (à supprimer après)
├── memory/           # Logs quotidiens
│   ├── 2026-02-01.md
│   └── 2026-02-02.md
├── skills/            # Skills spécifiques au workspace
└── canvas/            # Fichiers UI pour nodes (optionnel)

```

3.2 Création des Fichiers de Base

```

# Si les fichiers sont manquants
openclaw setup --workspace ~/.openclaw/workspace

```

3.3 Templates Recommandés

AGENTS.md :

```

# AGENTS.md - Instructions pour l'Agent

## Principes
- Être direct et utile, pas corporate
- Avoir des opinions, ne pas être un simple moteur de recherche
- Rester discret dans les groupes, ne pas répondre à tout

## Sécurité
- Ne jamais révéler de clés API
- Ne jamais effectuer de paiements
- Demander confirmation avant actions externes

## Mémoire
- Écrire les décisions importantes dans MEMORY.md
- Logger les erreurs dans .learnings/

```

SOUL.md :

```

# SOUL.md - Qui je suis

```

```
## Core Truths
- Genuinement utile, pas performativement utile
- Avoir des opinions
- Ressourceux avant de demander
- Gagner la confiance par la compétence

## Vibe
- Concis quand nécessaire, approfondi quand important
- Pas un drone corporate
- Juste... bon
```

4. Configuration Avancée du Gateway

4.1 Fichier de Configuration

Chemin : `~/openclaw/openclaw.json` (JSON5 - commentaires autorisés)

```
{
  // === AGENTS ===
  agents: {
    defaults: {
      // Workspace par défaut
      workspace: "~/openclaw/workspace",

      // Modèle par défaut
      model: "kimi-coding/kimi-for-coding",

      // Configuration sandbox
      sandbox: {
        mode: "non-main",      // "off" | "non-main" | "all"
        scope: "session",     // "session" | "agent" | "shared"
        workspaceAccess: "rw", // "none" | "ro" | "rw"
        docker: {
          image: "openclaw-sandbox:bookworm-slim",
          network: "bridge",  // "none" | "bridge" | "host"
          binds: [],
        },
      },
    },

    // Models disponibles avec alias
    models: {
      fast: { alias: "fast", provider: "anthropic", model: "claude-sonnet-4-5" },
      deep: { alias: "deep", provider: "anthropic", model: "claude-opus-4-5" },
    },
  },

  // Agents additionnels (multi-agent)
```

```
list: [
  {
    id: "main",
    default: true,
    name: "Assistant",
    workspace: "~/openclaw/workspace",
  },
  {
    id: "work",
    name: "Work Bot",
    workspace: "~/openclaw/workspace-work",
    model: "anthropic/claude-opus-4-5",
  },
],
},

// === SESSIONS ===
session: {
  dmScope: "main", // "main" | "per-peer" | "per-channel-peer"
  reset: {
    mode: "daily",
    atHour: 4,
    idleMinutes: 240,
  },
  mainKey: "main",
},

// === CHANNELS ===
channels: {
  whatsapp: {
    allowFrom: ["+336XXXXXXX"], // Whitelist DMs
    groups: {
      "*": { requireMention: true },
    },
  },
  telegram: {
    allowFrom: [],
    groups: {
      "*": { requireMention: false },
    },
  },
},

// === SKILLS ===
skills: {
  entries: {
    "nano-banana-pro": {
      enabled: true,
      apiKey: "${GEMINI_API_KEY}", // Variable d'env
    },
  },
},
},
```



```
// === ENV ===
env: {
  OPENROUTER_API_KEY: "${OPENROUTER_API_KEY}",
  shellEnv: {
    enabled: true,
    timeoutMs: 15000,
  },
},

// === BINDINGS (Multi-agent routing) ===
bindings: [
  { agentId: "main", match: { channel: "whatsapp" } },
  { agentId: "work", match: { channel: "telegram" } },
],
}
```

4.2 Modification de la Configuration

```
# Voir la config actuelle
openclaw gateway call config.get --params '{}'
```



```
# Modifier partiellement (merge)
openclaw gateway call config.patch --params '{
  "raw": "{ channels: { telegram: { groups: { \"*\": { requireMention: false } } } }",
  "baseHash": "<hash-from-config-get>",
  "restartDelayMs": 2000
}'
```



```
# Remplacer toute la config
openclaw gateway call config.apply --params '{
  "raw": "{ ... }",
  "baseHash": "<hash>",
  "note": "Mise à jour config"
}'
```

4.3 Variables d'Environnement

Ordre de précedence (haut → bas) :

1. Process environment
2. `.env` dans le répertoire courant
3. `~/.openclaw/.env` (global)

4. Bloc `env` dans `openclaw.json`

```
# Fichier ~/.openclaw/.env
OPENROUTER_API_KEY=sk-or-...
GEMINI_API_KEY=AI...
NOTION_API_KEY=ntn...
```

5. Développement de Skills

5.1 Qu'est-ce qu'un Skill ?

Un skill est un dossier contenant un fichier `SKILL.md` avec :

- Frontmatter YAML (métadonnées)
- Instructions détaillées pour l'agent
- Exemples d'utilisation

5.2 Structure d'un Skill

```
skills/mon-skill/
├─ SKILL.md           # Fichier principal (obligatoire)
├─ scripts/          # Scripts utilitaires (optionnel)
│   └─ setup.sh
└─ assets/           # Ressources (optionnel)
```

5.3 Template SKILL.md

```
---
name: mon-skill
description: Fait quelque chose d'utile avec l'API X
homepage: https://api-x.com
docs: https://docs.api-x.com
metadata:
  {
    "openclaw":
      {
        "emoji": "🔧",
        "requires": {
          "bins": ["curl", "jq"],
          "env": ["API_X_KEY"],
```

```
    "config": ["channels.enabled"]
  },
  "primaryEnv": "API_X_KEY",
  "install": [
    {
      "id": "brew",
      "kind": "brew",
      "formula": "api-x-cli",
      "bins": ["api-x"]
    }
  ]
}
}
}
---

# mon-skill

Utilise l'API X pour faire des choses incroyables.

## Prérequis

1. Obtenir une clé API : https://api-x.com/keys
2. Stocker la clé : `export API_X_KEY=votre_cle`

## Usage

```bash
Appel API basique
curl -H "Authorization: Bearer $API_X_KEY" \
 https://api-x.com/v1/endpoint
```

## Exemples

### Exemple 1 : Lister les items

```
curl "https://api-x.com/v1/items" \
 -H "Authorization: Bearer $API_X_KEY"
```

### Exemple 2 : Créer un item

```
curl -X POST "https://api-x.com/v1/items" \
 -H "Authorization: Bearer $API_X_KEY" \
 -H "Content-Type: application/json" \
 -d '{"name": "Nouvel item"}'
```

```
5.4 Installation de Skills

```bash
# Depuis ClawHub (registre public)
clawhub install notion
clawhub install gemini

# Mettre à jour tous les skills
clawhub update --all

# Synchroniser (scan + publish)
clawhub sync --all
```

5.5 Skills Locaux

```
# Créer un skill local
mkdir -p ~/.openclaw/skills/mon-skill
touch ~/.openclaw/skills/mon-skill/SKILL.md

# Ou dans le workspace (priorité plus haute)
mkdir -p ~/clawd/skills/mon-skill
touch ~/clawd/skills/mon-skill/SKILL.md
```

Ordre de priorité : `<workspace>/skills` > `~/.openclaw/skills` > `skills` intégrés

6. Gestion des Sessions & Agents

6.1 Types de Sessions

Type	Clé de session	Usage
Main (DM)	<code>agent:main:main</code>	Conversation principale 1:1
Groupe	<code>agent:main:whatsapp:group:</code> <code><id></code>	Chat de groupe isolé

Type	Clé de session	Usage
Cron	cron:<jobId>	Tâches planifiées
Webhook	hook:<uuid>	Intégrations web
Node	node-<nodeId>	Sessions depuis nodes

6.2 Gestion des Sessions

```
# Lister les sessions
openclaw sessions --json
openclaw sessions --active 60 # Active dans les 60 dernières minutes

# Voir l'historique d'une session
openclaw sessions history <sessionKey>

# Envoyer un message à une session
openclaw sessions send --sessionKey <key> --message "Hello"

# Status d'une session
openclaw session_status --sessionKey <key>
```

6.3 Multi-Agent

```
# Ajouter un agent
openclaw agents add work

# Lister les agents avec bindings
openclaw agents list --bindings

# Configurer le routing (exemple)
# Dans ~/.openclaw/openclaw.json :
{
  agents: {
    list: [
      { id: "personal", workspace: "~/.openclaw/workspace", default: true },
      { id: "work", workspace: "~/.openclaw/workspace-work" },
    ],
  },
  bindings: [
    { agentId: "personal", match: { channel: "whatsapp", accountId: "personal" } },
    { agentId: "work", match: { channel: "whatsapp", accountId: "work" } },
  ],
}
```

```
],  
}
```

6.4 Sub-agents (Sessions Spawn)

```
// Exécuter une tâche en arrière-plan  
sessions_spawn({  
  task: "Analyser ce codebase et créer un rapport",  
  runTimeoutSeconds: 300  
})
```

Gestion des sub-agents :

```
# Lister les sub-agents actifs  
/subagents list  
  
# Voir les logs  
/subagents log <runId>  
  
# Arrêter un sub-agent  
/subagents stop <runId>  
  
# Envoyer un message  
/subagents send <runId> "Continue avec l'étape 2"
```

7. Intégrations Channels

7.1 WhatsApp

```
# Connexion (scan QR)  
openclaw channels login whatsapp  
  
# Approbation de pairing  
openclaw pairing list whatsapp  
openclaw pairing approve whatsapp <code>
```

Configuration :

```
{  
  channels: {  
    whatsapp: {
```

```

    allowFrom: ["+33612345678"],
    groups: {
      "*": { requireMention: true },
      "1203630...@g.us": { requireMention: false },
    },
  },
},
},
}

```

7.2 Telegram

```

# Créer un bot via @BotFather, obtenir le token
# Le wizard peut le configurer, ou manuellement :

```

Configuration :

```

{
  channels: {
    telegram: {
      botToken: "${TELEGRAM_BOT_TOKEN}",
      allowFrom: [],
      groups: { "*": { requireMention: false } },
    },
  },
}

```

7.3 Discord

```

{
  channels: {
    discord: {
      botToken: "${DISCORD_BOT_TOKEN}",
      guilds: {
        "*": { requireMention: true },
      },
    },
  },
}

```

7.4 WebChat / Dashboard

Accessible localement : <http://127.0.0.1:18789/>

Pour accès distant :

- SSH tunnel : `ssh -N -L 18789:127.0.0.1:18789 user@host`
 - Tailscale : voir [Tailscale docs](#)
-

8. Sécurité & Sandbox

8.1 Politique de Sécurité par Défaut

Action	Comportement
<code>exec</code>	Approbation requise (ask mode)
<code>write / edit</code>	Sans restriction dans le workspace
<code>browser</code>	Sans restriction (loopback)
Clés API	Jamais affichées dans les réponses

8.2 Modes Sandbox

```
{
  agents: {
    defaults: {
      sandbox: {
        // Quand sandboxer ?
        mode: "non-main", // "off" | "non-main" | "all"

        // Scope des containers
        scope: "session", // "session" | "agent" | "shared"

        // Accès workspace
        workspaceAccess: "rw", // "none" | "ro" | "rw"

        docker: {
          // Image Docker
          image: "openclaw-sandbox:bookworm-slim",

          // Réseau
          network: "none", // "none" | "bridge" | "host"
        }
      }
    }
  }
}
```



```

    // Bind mounts additionnels
    binds: [
        "/home/user/projects:/projects:ro",
        "/var/run/docker.sock:/var/run/docker.sock",
    ],

    // Commande de setup (une fois)
    setupCommand: "apt-get update && apt-get install -y nodejs",

    // Variables d'env dans le sandbox
    env: {
        NODE_ENV: "production",
    },
},
},
},
},
}

```

8.3 Construire l'Image Sandbox

```

# Image de base
scripts/sandbox-setup.sh

# Image avec navigateur
scripts/sandbox-browser-setup.sh

```

8.4 Elevated Mode

```

# Activer le mode élevé (bypass certaines restrictions)
/elevated on

# Commandes disponibles en élevé
/elevated full # Skip exec approvals

```

Configuration :

```

{
  tools: {
    elevated: {
      ask: "on-miss", // "off" | "on-miss" | "always"
      allow: ["+336XXXXXXX"], // Qui peut utiliser elevated
    },
  },
}

```

```
},  
}
```

8.5 Clés API & Secrets

Règles d'or :

-  Jamais de clés en dur dans le code
-  Utiliser `${VAR_NAME}` dans les configs
-  Stocker dans `~/.openclaw/.env`
-  Utiliser environment variables

```
# Vérifier qu'aucune clé ne fuite avant commit  
grep -r "sk-" ~/.openclaw/ 2>/dev/null  
grep -r "AQ\." ~/.openclaw/ 2>/dev/null
```

9. Automatisations & Cron

9.1 Types de Jobs

Schedule	Description
<code>at</code>	One-shot à une date/heure
<code>every</code>	Intervalle fixe (ms)
<code>cron</code>	Expression cron (5 champs)

9.2 Exemples de Jobs

```
// Job one-shot (rappel)  
cron({  
  action: "add",  
  job: {  
    name: "Rappel réunion",  
    schedule: { kind: "at", atMs: Date.now() + 20 * 60 * 1000 },  
  },  
})
```

```
    payload: {
      kind: "systemEvent",
      text: "🕒 Rappel : Réunion dans 10 minutes !"
    },
    sessionTarget: "main"
  }
})

// Job récurrent (tous les jours à 9h)
cron({
  action: "add",
  job: {
    name: "Daily check",
    schedule: { kind: "cron", expr: "0 9 * * *", tz: "Europe/Paris" },
    payload: {
      kind: "agentTurn",
      message: "Faire un check des emails et du calendrier"
    },
    sessionTarget: "isolated"
  }
})

// Intervalle (toutes les heures)
cron({
  action: "add",
  job: {
    name: "Hourly sync",
    schedule: { kind: "every", everyMs: 3600000 },
    payload: { kind: "systemEvent", text: "Sync automatique" },
    sessionTarget: "main"
  }
})
```

9.3 Gestion des Jobs

```
# Lister les jobs
openclaw cron list

# Exécuter immédiatement
openclaw cron run <jobId>

# Voir l'historique
openclaw cron runs <jobId>

# Supprimer
openclaw cron remove <jobId>
```

9.4 Heartbeat vs Cron

Usage	Recommandation
Vérifications groupées (emails, calendrier)	Heartbeat
Timing précis (9h00 pile)	Cron
Isolation complète	Cron
Besoin de contexte conversationnel	Heartbeat
Rappel one-shot	Cron

10. Bonnes Pratiques & Debugging

10.1 Commandes de Debug

```
# Status complet
openclaw status --all
openclaw status --deep

# Logs
openclaw logs
openclaw logs --follow

# Docteur (diagnostics)
openclaw doctor
openclaw doctor --fix # Auto-réparation

# Expliquer la sandbox actuelle
openclaw sandbox explain
```

10.2 Slash Commands Utiles

Commande	Description
<code>/status</code>	Voir le statut courant

Commande	Description
<code>/context list</code>	Contenu du contexte actuel
<code>/context detail</code>	Détail par fichier/outil
<code>/model <name></code>	Changer de modèle
<code>/think <level></code>	Niveau de réflexion
<code>/verbose on/off</code>	Mode verbeux
<code>/reset</code> OU <code>/new</code>	Nouvelle session
<code>/compact</code>	Compacter le contexte
<code>/stop</code>	Arrêter l'exécution
<code>/subagents list</code>	Voir les sub-agents
<code>/usage</code>	Statistiques d'utilisation

10.3 Debugging Checklist

****Problème : L'agent ne répond pas****

1. ``openclaw health`` - Gateway en ligne ?
2. ``openclaw pairing list <channel>`` - Pairing approuvé ?
3. Vérifier les logs : ``openclaw logs --follow``
4. Vérifier ``allowFrom`` dans la config

****Problème : Erreur de configuration****

1. ``openclaw doctor`` - Voir les erreurs
2. ``openclaw doctor --fix`` - Auto-réparer
3. Vérifier la syntaxe JSON5 (virgules, guillemets)

****Problème : Tool bloqué****

1. Vérifier ``tools.allow`` / ``tools.deny``
2. Vérifier ``sandbox.mode`` et ``workspaceAccess``
3. ``openclaw sandbox explain`` - Voir les règles effectives

```
### 10.4 Backup & Migration

```bash
Backup du workspace
cd ~/.openclaw/workspace
git init
git add .
git commit -m "Backup workspace"
git push origin main

Backup des sessions (optionnel)
cp -r ~/.openclaw/agents ~/backup/agents

Migration vers nouvelle machine
1. Cloner le repo workspace
git clone <repo> ~/.openclaw/workspace

2. Copier les credentials (si nécessaire)
OAuth: ~/.openclaw/credentials/oauth.json
Sessions: ~/.openclaw/agents/<id>/sessions/

3. Reconfigurer les channels
openclaw channels login
```

## 10.5 Performance

```
Voir l'utilisation des tokens
/usage full

Compacter manuellement
/compact "Résumer les décisions importantes"

Changer de modèle pour plus de vitesse
/model kimi-coding/kimi-for-coding
```

## 10.6 Fichier .gitignore Recommandé

```
OpenClaw workspace gitignore
.DS_Store
.env
**/*.key
**/*.pem
**/secrets*
node_modules/
*.log
```

## Exemple Complet : Setup Dev

```
1. Installation
curl -fsSL https://openclaw.bot/install.sh | bash

2. Onboarding
openclaw onboard --install-daemon

3. Créer le workspace de dev
mkdir -p ~/clawd
cd ~/clawd

4. Créer les fichiers de base
cat > AGENTS.md << 'EOF'
AGENTS.md - Workspace Dev

Principes
- Toujours utiliser git pour versionner
- Écrire les learnings dans .learnings/
- Demander confirmation avant actions destructrices
EOF

cat > SOUL.md << 'EOF'
SOUL.md

Identité
- Assistant de développement
- Pragmatique et efficace
- Pose des questions si ambigu
EOF

5. Configurer les skills
cd ~/clawd
clawhub install notion
clawhub install kimi-cli

6. Créer le .env
mkdir -p ~/.openclaw
cat > ~/.openclaw/.env << 'EOF'
NOTION_API_KEY=ntn_xxx
KIMI_API_KEY=xxx
EOF

7. Vérifier
openclaw status
openclaw health

8. Premier test
openclaw message send --target +336XXXXXXX --message "OpenClaw est configuré !"
```



## Ressources

- **Documentation officielle** : <https://docs.openclaw.ai>
  - **GitHub** : <https://github.com/openclaw/openclaw>
  - **ClawHub** (Skills) : <https://clawhub.com>
  - **Discord communauté** : <https://discord.com/invite/clawd>
- 

*Dernière mise à jour : Février 2026 Version OpenClaw : 2026.x*